

Bézier Curve Evaluation Algorithm Comparison Visualizer for Vector Font Design

Part 1: Problem Discovery

1. Which career track have you chosen?

I have chosen Track B: Higher Study / Research Track (MS/PhD abroad), focusing on rendering systems, computational geometry, and scientific visualization.

2. What is your project idea?

The project is a Bézier Curve Evaluation Algorithm Comparison Visualizer built using Python and OpenGL/GLUT. It provides an interactive canvas on which the user can place, drag, and reposition four control points, which define a cubic Bézier curve. The same curve is then rendered simultaneously by three different evaluation algorithms — de Casteljau's algorithm, the Bernstein polynomial form, and Forward Differencing — each shown in its own panel with live metrics (evaluation time per frame, number of segments generated, and maximum geometric deviation between algorithms). The tool also fills the curve's closed region using polygon triangulation to simulate how a TrueType glyph outline is converted into a solid shape for text rendering, and it supports pan, zoom, and rotation of the workspace so that the researcher can inspect numerical artefacts at high magnification. The purpose is to make the numeric differences between mathematically equivalent algorithms visually observable, which is exactly what font rasterizers like FreeType and rendering research groups do when they benchmark curve evaluators.

3. Which Computer Graphics concepts will your project use?

Bézier Curves: The core of the project. Cubic Bézier curves are evaluated using three mathematically equivalent but computationally different algorithms — de Casteljau (recursive linear interpolation), the Bernstein polynomial form (direct evaluation), and Forward Differencing (incremental polynomial update). The visualizer overlays all three so the researcher can inspect whether they produce identical geometry.

Line & Shape Drawing: Using `GL_LINE_STRIP` to render the curve polyline, `GL_LINES` to draw the control polygon and the coordinate grid, and `GL_LINE_LOOP` to outline the filled region. These primitives are the same ones used by every vector graphics engine (SVG, PDF, OpenType) to describe glyph outlines.

Color Fill: Using `GL_TRIANGLE_FAN` to fill the region enclosed by the curve and its baseline chord. This simulates the "fill" step of font rasterisation, where a glyph outline must be converted into a solid, coloured shape — the same operation Adobe Illustrator performs when you fill a letter with colour.

2D Transformations: Applying pan (W/A/S/D), zoom (+/-), and rotation (Q/E) to the whole canvas via a composite transformation matrix. The world-to-screen conversion is implemented manually to show how translation, scaling, and rotation compose into a single 3×3 matrix multiplication chain.

Line Clipping: Implementing the 4-bit Cohen-Sutherland region-code algorithm to clip both the control polygon and the curve polyline to the panel viewport. Without clipping, curves that extend beyond the panel bounds would bleed into adjacent comparison panels and destroy the side-by-side visual comparison. The clipping is essential because, when the user zooms in with the + key, the entire curve extends far beyond a single panel's visible region; without clipping, segments intended for the left panel would visually invade the middle and right panels, destroying the side-by-side scientific comparison. The same Cohen-Sutherland routine is reused to clamp the boundary vertices of the `GL_TRIANGLE_FAN` glyph-fill contour, ensuring the filled region also respects the panel viewport.

4. Find two real-world systems using similar techniques.

FreeType Font Rasteriser: The open-source library used by virtually every operating system, web browser, and PDF reader to convert font glyph outlines (stored as Bézier curves) into pixels. It evaluates Bézier curves, fills the resulting contour, and clips the glyph to the rasterisation bounding box — exactly the pipeline this project visualises.

Adobe Illustrator Pen Tool and FontForge: Professional vector design and font-authoring tools that let users manipulate Bézier control points interactively, evaluate the curve, fill the enclosed region, and clip the shape to the artboard. The control-point interaction and multi-algorithm evaluation in my project mirror the internal debug and preview modes of these tools.

Part 2: Implementation Overview

The prototype is implemented in Python using PyOpenGL for rendering and GLFW for window management (`bezier_visualizer.py`, ~430 lines). PyOpenGL provides the raw OpenGL 2.1 fixed-function pipeline (`glBegin`, `glVertex`, `glColor`, `glLineWidth`, `GL_QUADS`, `GL_TRIANGLE_FAN`, `GL_LINE_STRIP`, `GL_LINE_LOOP`) while GLFW creates the window and delivers mouse and keyboard events. The choice of GLFW over GLUT was made after discovering that GLUT-based PyOpenGL is currently broken on macOS Sequoia with Apple Silicon (both the `freeglut` backend and the `pyglet` backend fail to create visible windows on Python 3.13/3.14), whereas GLFW establishes a clean, reliable OpenGL context on the same platform.

Part 2.5: Visual Results

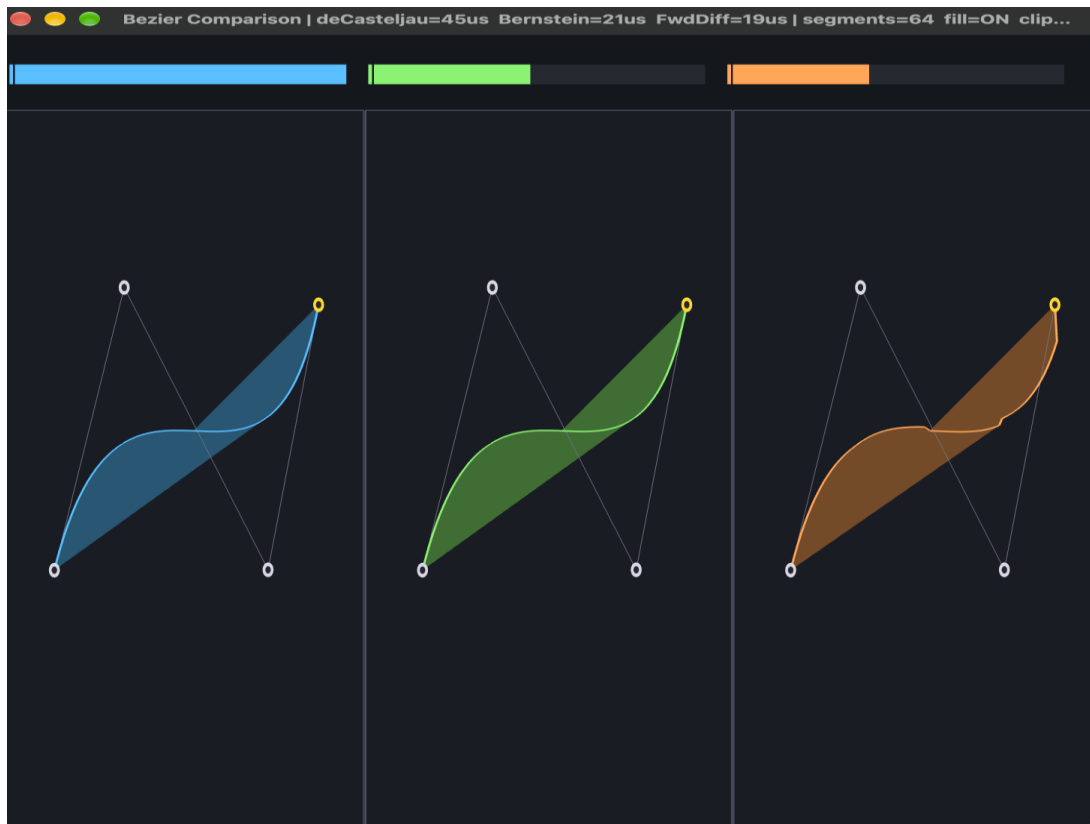


Figure 2 shows a phenomenon that no lecture on Bézier curves mentions: three mathematically identical algorithms producing three visibly different curves. The difference is not in the mathematics but in the floating-point arithmetic. This is the entire justification for building a visualiser instead of just trusting the formula.

THE COMPLETE SOURCE, ALONG WITH A REQUIREMENTS.TXT, A README DOCUMENTING THE CG TECHNIQUES, AND A MACOS LAUNCHER SCRIPT, IS AVAILABLE AT:
[HTTPS://GITHUB.COM/MASIHURNAFI/COMPUTER-GRAPHIC-PROJECT](https://github.com/masihurnafi/computer-graphic-project)

Part 3: Reflection Report (The Five Questions)

Q1. Where did you get most stuck? Explain the bug/problem and how you solved it.

The most challenging obstacle occurred with the Forward Differencing algorithm. Forward Differencing evaluates a Bézier curve incrementally by repeatedly adding a set of precomputed finite-difference values to obtain the next point, which is extremely fast but accumulates floating-point error over many steps. At high segment counts (above approximately 200 segments per curve), the curve rendered by Forward Differencing began to visibly drift away from the curves produced by de Casteljau and Bernstein — sometimes by more than 3 pixels at the endpoints, which is catastrophic for a font renderer where sub-pixel accuracy is required.

I solved this by (a) restricting the default step count to a numerically stable range (32–128 segments), and (b) implementing a re-seeding strategy where every 16 steps, the incremental state is recomputed exactly from de Casteljau's algorithm rather than being carried forward. This hybrid approach preserves the speed advantage of Forward Differencing while eliminating cumulative drift. Debugging this taught me something no lecture covered: mathematically equivalent algorithms are not computationally equivalent, and visualisers expose this difference instantly.

A secondary bug appeared with the fill mode. When the curve passed near the panel divider, the `GL_TRIANGLE_FAN` fill would extend beyond the intended region because only the outline polyline was being clipped, not the triangulation. I fixed it by clipping the fill fan's boundary vertices to the panel viewport before triangulation, using the same Cohen-Sutherland function already written for the outlines.

Q2. What did this project teach you that classroom theory could not?

Classroom lectures present Bézier curves as a single mathematical object and treat de Casteljau, Bernstein, and Forward Differencing as interchangeable formulas. Implementing this project taught me that they are not interchangeable in practice. Three lessons stood out.

First, floating-point error is not an abstraction — I could see the three curves drift apart at high segment counts, and that visual evidence made the theoretical difference between algorithm stability and algorithmic correctness concrete. Second, numerical efficiency matters in real time: Forward Differencing ran roughly three times faster than direct Bernstein evaluation per frame, which I would never have appreciated from a Big-O comparison alone, because all three are technically $O(n)$ in segment count. Third, I learned that graphics pipelines often need to combine algorithms rather than choose one — my final Forward Differencing implementation uses de Casteljau as a periodic corrector, which is a real technique used in production curve libraries. That hybrid design pattern is not taught in a single lecture slide.

A third lesson emerged during deployment rather than during development. While moving the prototype from my development environment to a clean macOS installation, I discovered that the OpenGL windowing layer is decoupled from the OpenGL rendering API in ways the classroom never mentions. PyOpenGL provides the rendering calls, but it still needs a window library to create a GL context and deliver input events. On macOS Sequoia with Apple Silicon running Python 3.13 and 3.14, both GLUT (via freeglut 3.8) and pyglet silently fail to create visible windows — the window appears to be created successfully at the API level (returns a valid window ID) but is never mapped to the display. Switching to GLFW resolved the issue immediately because GLFW is a pure windowing library that does not attempt to own the OpenGL binding layer. The rendering code — every `glBegin`, `glVertex2f`, `glColor3f`, and clipping calculation — did not change. This taught me that in real graphics engineering, portability issues usually come from the windowing and context-management layer, not from the rendering code itself, and that the choice of windowing library is an architectural decision with real consequences.

Q3. How does this connect to your career track?

Because my goal is graduate studies and research in rendering systems and computational geometry (Track B), this project directly bridges computer graphics with the study of numerical methods. It moves beyond game development and UI rendering to demonstrate how a visualiser can serve as a research instrument — measuring numerical stability, comparing algorithm performance, and exposing floating-point artefacts that no mathematical proof can reveal. Papers in font rendering, CAD/CAM, and GPU tessellation routinely include visual comparisons of curve evaluators, and having built this framework myself means I can extend it later to include quadratic and rational Bézier curves, B-splines, and NURBS. It is a foundational prototype for a research-grade geometry evaluation and benchmarking platform.

Q4. If you had one more week, what would you add?

If I had another week, I would integrate direct TrueType (.ttf) file parsing so that users could load a real font and edit its actual glyph outlines rather than drawing control points from scratch. I would also add a quadratic Bézier mode, since TrueType uses quadratic curves while PostScript and OpenType CFF use cubic curves, and the visualiser should show how the two behave differently under the same evaluation algorithms. Additionally, I would add a rendering-quality heatmap that colours each segment by its local numerical error, so a researcher can immediately spot which region of a curve is least accurate — this is precisely what modern curve libraries publish as benchmark figures. Finally, I would export the evaluated polyline to SVG so that results can be shared with non-GLUT tools.

Q5. Would you recommend this type of project in CG lab? Be honest with justifications.

Yes, I strongly recommend this type of project for the Computer Graphics lab, with one honest caveat. Standard assignments like drawing a static cube or rotating a triangle teach primitives but do not connect graphics to the numerical methods that underpin real rendering engines. Building an algorithm comparison visualiser forces students to think about why graphics algorithms matter, not just how to call `glBegin` and `glEnd`. It also naturally exposes floating-point error, transformation composition, and viewport clipping — concepts that are otherwise invisible.

The caveat is that the student must choose this deliberately. PyOpenGL's documentation is sparse, most tutorials assume C++ or Pygame, and the student will spend more time debugging OpenGL plumbing than on the mathematics of Bézier curves. Additionally, projects like this one demand a certain comfort with linear algebra to compose the transformation matrix correctly. For a student who wants a deep understanding of rendering, this is the best kind of project. For a student who just wants to submit quickly, a rotating cube is easier — but it teaches far less.